

EX.NO:5	Use of Process, File, Stat and Directory system Calls
DATE: 07-07-26	

(A) FILE SYSTEM CALL

AIM:

To write a C++ program to exhibit the concept of file system calls and its use.

ALGORITHM:

1. Start the program.
2. Import the necessary header files (<iostream>, <fcntl.h>, <unistd.h>, <string.h>).
3. Declare necessary variables for the file descriptor (fd) and data buffer.
4. Declare a static character array Message.
5. Assign the string "HELLOWORLD" to Message.
6. Declare a character buffer to read data.
7. Open the file using the open() system call with read/write and creation flags (O_CREAT | O_RDWR).
8. Check if fd == -1. If true, print an error message and exit.
9. Print "File is opened for read/write access" if the file opens successfully.
10. Write the message into the file using the write() system call.
11. Reposition the file offset pointer to the beginning using lseek(fd,0L, 0).
12. Read the content from the file to verify it.
13. Print "Message written to file" along with the content.
14. Else print error and close the file.
15. Else print file content exists.
16. Stop the program.

PROGRAM:

INPUT:

```
#include <iostream>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
using namespace std;
static char message[] = "Hello World";
int main()
{   int fd;
    char buffer[80];
    cout << "Mitun 25IT371\n";
    fd = open("df.dat", O_RDWR | O_CREAT | O_EXCL, S_IREAD | S_IWRITE);
    if (fd != -1)
    {
        cout << "Datafile df.dat opened for read/write access\n";
        write(fd, message, sizeof(message));
        lseek(fd, 0L, 0);
        if (read(fd, buffer, sizeof(message)) == sizeof(message))
        {
            cout << buffer << " was written to datafile.dat\n";
        }
        else
        {
            cout << "***error reading datafile.dat***\n";
        }
        close(fd);
    }
    else
    {
        cout << "ds.dat already exists\n";
    }
    exit(0);
}
```

OUTPUT:

```
Mitun 25IT371  
Datafile df.dat opened for read/write access  
Hello World was written to datafile.dat
```

```
Mitun 25IT371  
ds.dat already exists
```

RESULT:

Thus the use of file system call using c program has been illustrated and executed successfully.

A) USE OF DIRECTORY SYSTEM

CALLS AIM:

To write a program to illustrate the concept of Directory System call.

ALGORITHM:

1. START
2. Import necessary header files and variables
3. Search()
4. Create a pointer for DIR
5. If file is empty or directory is empty
6. Print unable to open directory
7. Get the name of file to be searched
8. while dir is empty or NULL
9. Check for the file
10. add 1 to flag
11. if flag is 1
12. print file is found
13. else
14. Print file is not found
15. main()
16. get name of directory
17. Search(name)
18. STOP

PROGRAM:

INPUT:

```
#include <iostream>
#include <dirent.h>
#include <stdlib.h>
#include <string.h>
using namespace std;
```

```

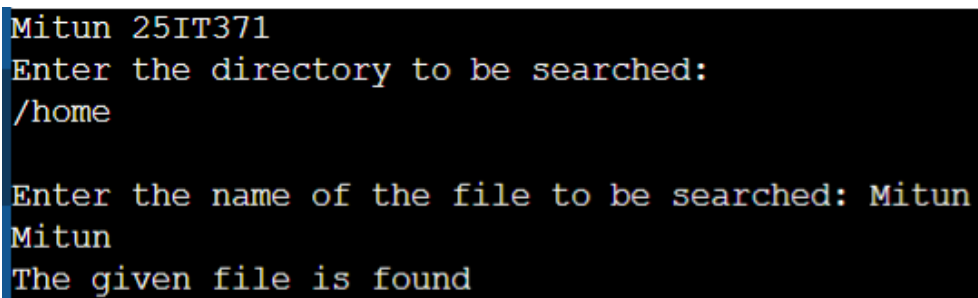
void sea(char *dname)
{
    DIR *dir;
    struct dirent *ent;
    int flag = 0;
    char a[15];
    if ((dir = opendir(dname)) == NULL)
    {
        cout << "\nUnable to open directory";
        exit(1);
    }
    cout << "\nEnter the name of the file to be
searched: ";
    cin >> a;
    while ((ent = readdir(dir)) != NULL)
    {
        if (!strcmp(a, ent->d_name)){
            cout << ent->d_name;
            flag++;
        }
    }
    if (flag == 1)
    {
        cout << "\nThe given file is found\n";
    }
    else
    {
        cout << "\nFile not found";
    }
    if (closedir(dir) != 0)
    {
        cout << "Unable to close directory";
    }
}

int main()
{
    char dirname[25];
    cout << "Mitun 25IT371";
}

```

```
cout << "\nEnter the directory to be  
searched:\n";  
cin >> dirname;  
sea(dirname);  
return 0;  
}
```

OUTPUT:



```
Mitun 25IT371  
Enter the directory to be searched:  
/home  
  
Enter the name of the file to be searched: Mitun  
Mitun  
The given file is found
```

RESULT:

Thus the use of directory system call using c program has been illustrated and executed successfully.

B) USE OF PROCESS SYSTEM CALLS

AIM:

To write a program to exhibit the concept of Process system calls.

ALGORITHM:

1. Start
2. Import necessary files (header files)
3. Declare pid
4. Initiate pid = fork()
5. if (Pid < 0)
6. Print fork cannot be created
7. else if (Pid == 0)
8. Print process id of child and parent
9. else
10. Print process id of parent and grandparent.

PROGRAM:

INPUT:

```
#include <iostream>

#include <unistd.h>

#include <stdlib.h>

using namespace std;

int main()
{
    int pid;

    pid = fork();

    cout << "Mitun 25IT371\n";

    if (pid < 0)
```

```
{  
    printf("Fork failed\t");  
    printf("The process id of the parent is %d", getpid());  
    exit(-1);  
}  
else if (pid == 0)  
{  
    execlp("/home/ls", "ls", NULL);  
    cout << "The process id of the child is " << getpid();  
    cout << "\nThe process id of the parent is " << getppid();  
}  
else  
{  
    cout << "The process id of the parent is " << getpid();  
    cout << "\nThe process id of the grandparent is " << getppid();  
}  
return 0;  
}
```


OUTPUT:

```
Mitun 25IT371
Mitun 25IT371
The process id of the parent is 658
The process id of the child is 662
The process id of the parent is 658The process id of the grandparent is 657
```

RESULT:

Thus the use of process system call using c program has been illustrated and executed successfully.

C) USE OF STAT SYSTEM CALL

AIM:

To write a program to exhibit the concept of stat system call and its use.

ALGORITHM:

1. Start
2. Declare a structure and a variable s
3. Declare necessary variables
4. If (stat("test",&s)==-1)
5. Show a error
6. exit
7. Compute size of the input files print the file size

PROGRAM:

INPUT:

```
#include <iostream>
#include <sys/stat.h>
#include <stdlib.h>
using namespace std;
int main()
{
    char file[20];
    struct stat s;
    int a, b;
    cout << "Mitun 25IT371\n";
```

```
cout << "Enter the filename:\n";
cin >> file;
if (stat(file, &s) == -1)
{
    cout << "The file is not found";
    exit(0);
}
else
{
    a= s.st_blksize;
    b= s.st_size;
    cout << "\nThe allocated block size is : " << a;
    cout << "\nThe actual size is : " << b;
}
return 0;
}
```

OUTPUT:

```
Mitun 25IT371
Enter the filename:
Mitun

The allocated block size is : 4096
The actual size is : 92
```

```
main.cpp  Mitun  ⋮
1  My name is Mitun
2  I love to watch movies
3  I like to play games
4  Badminton is my favourite sport
```

RESULT:

Thus the use of stat system call using c program has been illustrated and executed successfully.